An intent-based solution for network isolation in Kubernetes

F. Pizzato, D. Bringhenti, R. Sisto, F. Valenza

SecSoft, xxth June 2024

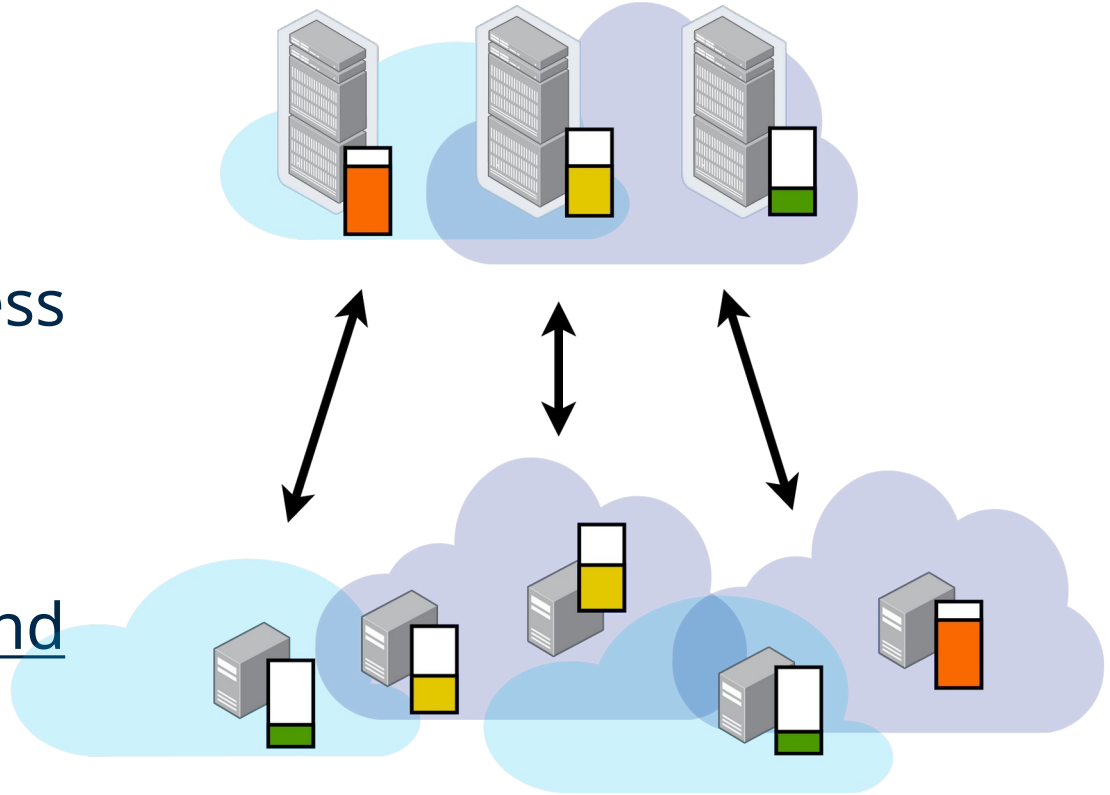
Politecnico di Torino

netgroup

DAUIN

Liquid Computing

- Millions of heterogeneous computing devices interconnected and often underutilized. Waste of resource and computing power.
- **Liquid computing** is a novel cloud computing paradigm that allows a seamless sharing of heterogeneous resources between different tenants, creating a seamless pool of resources.
- However, this raises <u>novel management and security problems</u>!



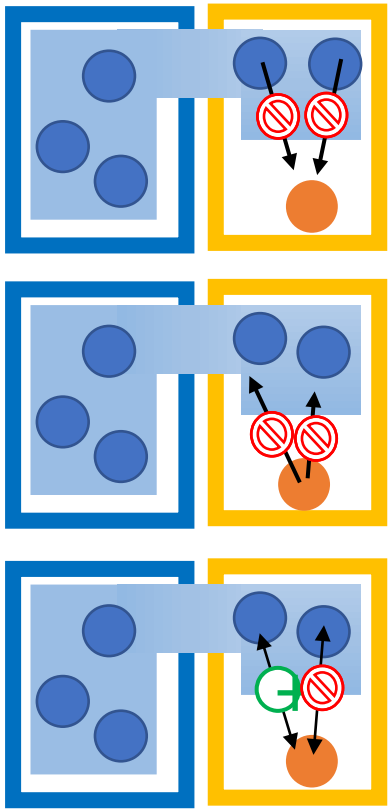
Network isolation

A significant problem is **network isolation**, which involves protecting the dynamic virtual boundaries extending over different physical machines and administrative domains. It is a multi-purpose problem:

1. *Protect the hosting cluster against potential harm from guest applications deployed by a different tenant*

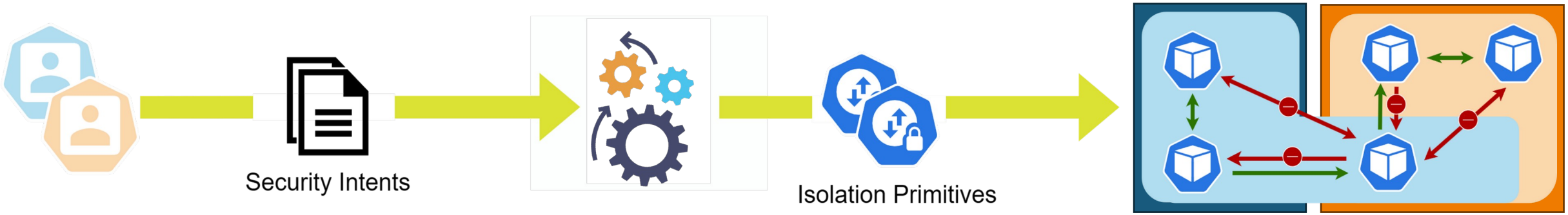
2. *Protect guest applications against potential stealing of data or code or unauthorized interference from the host*

3. *Allow application owners to precisely specify which interactions their applications may have with the environment*



Proposal

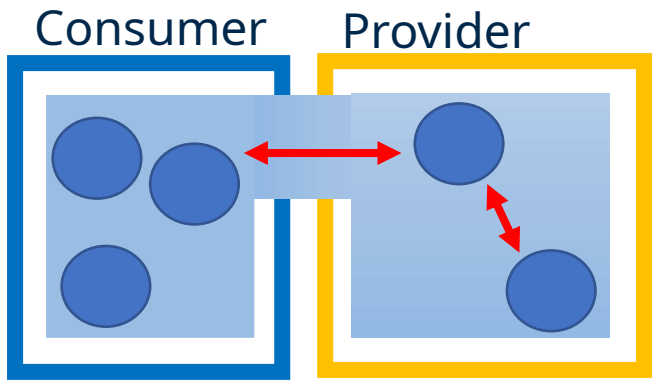
- Security orchestrator to automate the configuration of network isolation primitives across a multi-domain and multi-tenant cloud environment.
- Intent-driven declarative solution, i.e., users define their requirements in terms of prohibited and desired network communications through a user-friendly language.
- Allows the definition of finely-grained isolation policies to implement common security principles, e.g., zero trust and least privilege.



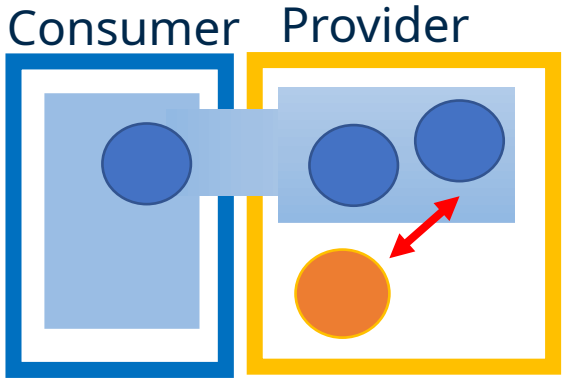
Use cases for Liquid Computing

In this “liquid” environment, resources could be acquired or rented between different users. The consumer uses resources of a remote cluster, the provider supplies resources to other clusters.

Our solution to the communications happening in the use-cases specific to liquid computing, such as:



Workloads are offloaded to a remote cluster to absorb load spiked, i.e., Cloud bursting.



Workloads are offloaded to a remote cluster to locally access data, i.e., data gravity, or accessing a service.

Different goals and different intents

Consumer

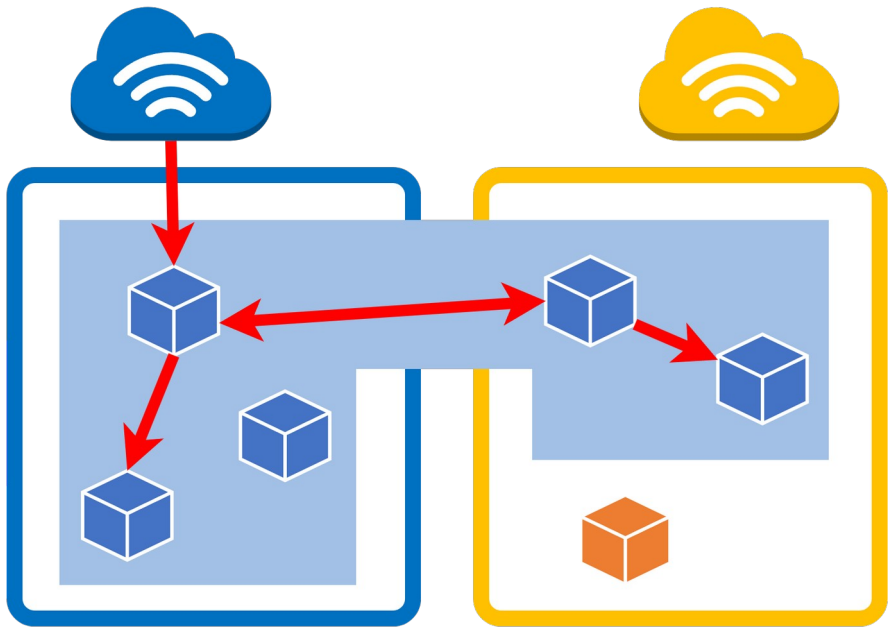
- the one who is buying some resource from remote clusters
- wants to safeguard communications within its local cluster (**Private Intents**) and the communications among resources offloaded to remote clusters (**Request Intents**)

Provider

- the one who is offering some of its resources to remote clusters
- wants to limit communications involving the hosted resources on the one hand, its own services or the external network on the other hand (***Authorization Intents***)

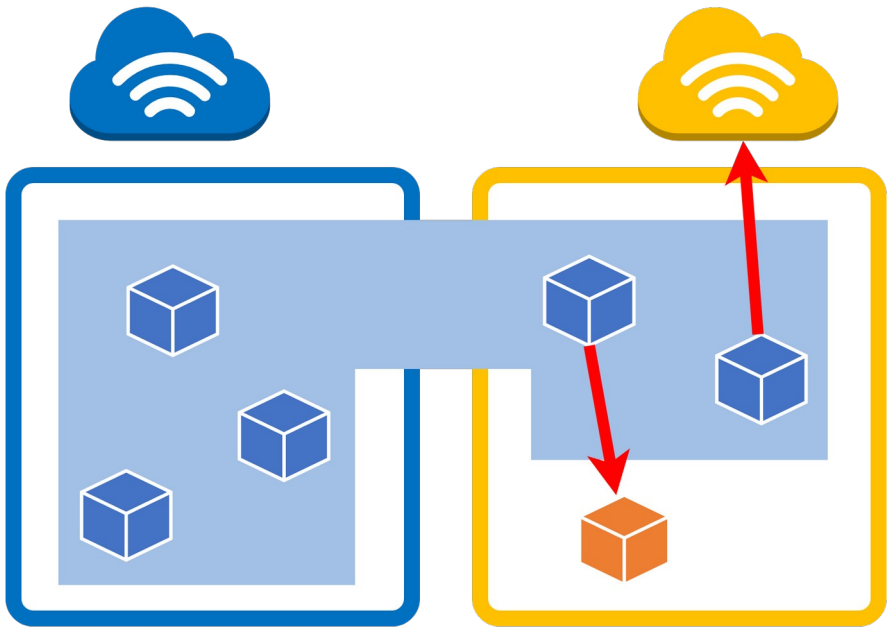
Private Intents

- They are related to communications happening within the virtual cluster (i.e., intra-virtual cluster), involving both the local and remote resources.
- These are <u>not</u> subject to the authorization of the host(s). The principle is that each user should have unlimited control over his own resources



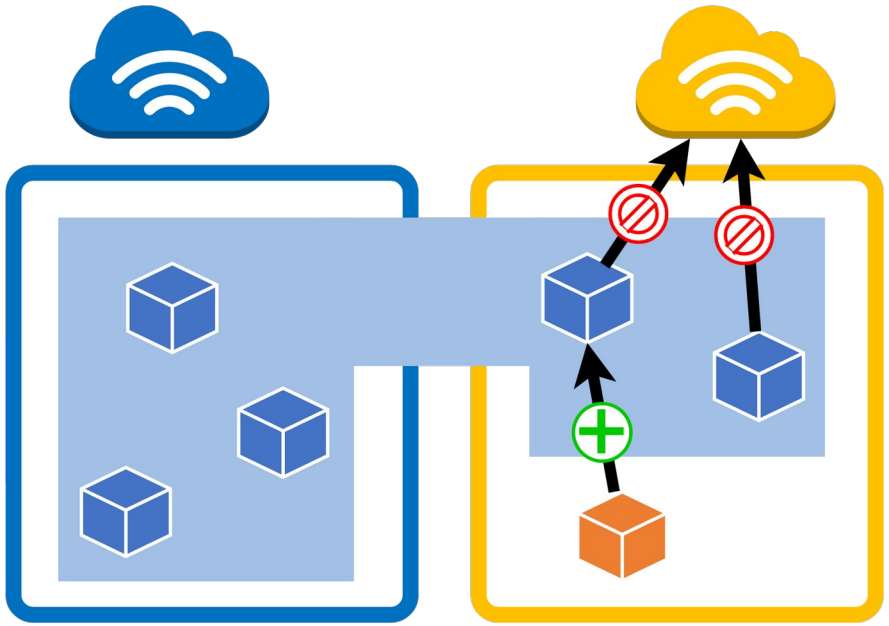
Request Intents

- They are related to communications crossing the virtual border of an extended cluster and could be configured to target services offered by the hosting cluster or addresses on the external network.
- The consumer could tune some parameters to refine his will in terms of isolation with respect to the hosting cluster (e.g., to accept or refuse to be monitored by the provider)



Authorization Intents

- They represent the authorization policies defined by a user with respect to any possible guest, to regulate the inter-virtual cluster communications of each hosted cluster.
- They target two communication types:
  - **DeniedCommunications** must always be blocked or filtered for all hosts.
  - **MandatoryCommunications** must be injected to all hosts (e.g., to whitelist some monitoring probes)



Intents' Format

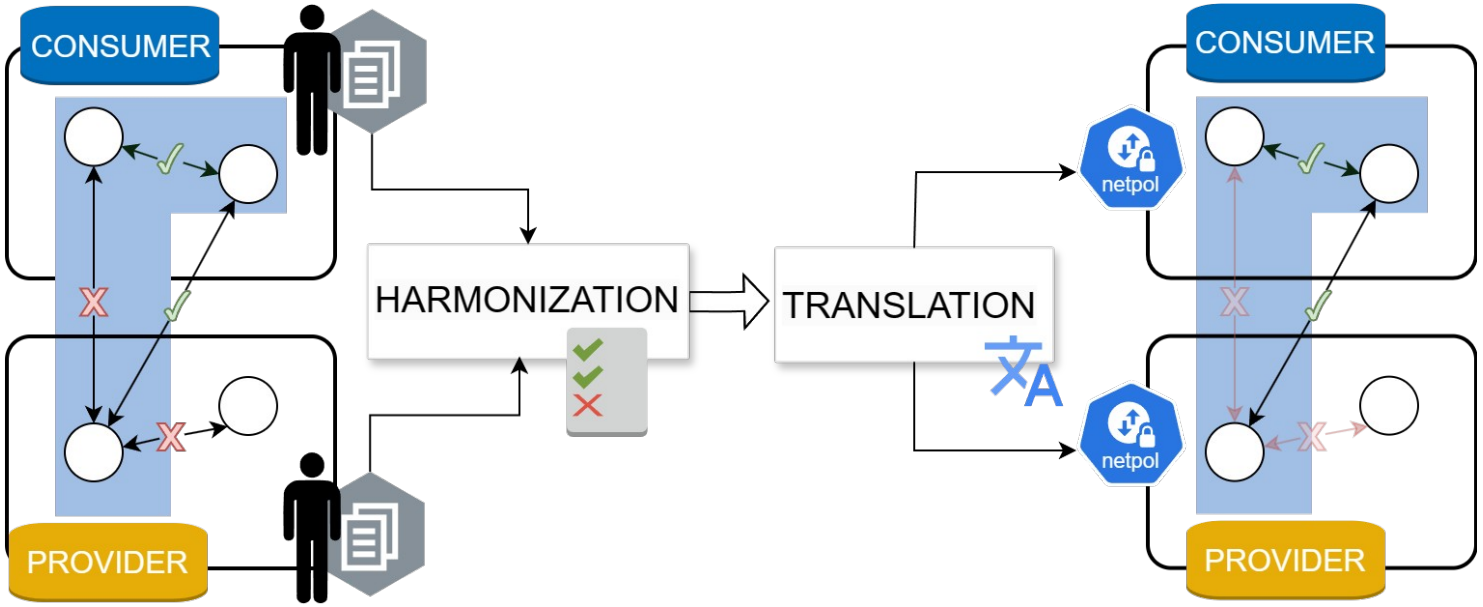
"from SRC to DST, protocol[:port[-endPort]]"

- *SRC* and *DST* can be either a Pod, or a group of Pods, with the same label and an associated Namespace, or an CIDR address. Could use wildcard.
- *Protocol* can be any transport protocol (TCP, UDP, STCP, etc.) or the value ALL to represent al of them.
- *Port* can be a port or a range of ports.

- Request and Private intents can be expressed only in whitelisting, so as to be compliant with the default behavior of Kubernetes Network Policy.
- Authorization intents allow for more expressiveness. The *DeniedCommunications* are expressed in blacklisting, and conversely the *MandatoryCommunications* are simply allowed communications.

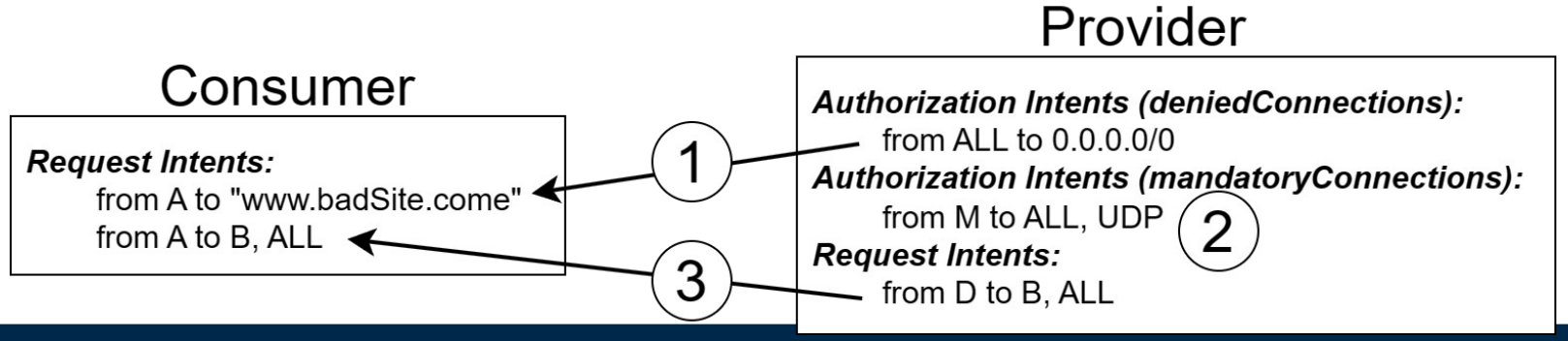
Workflow

- The consumer and the provider should communicate their intents during the resource acquisition phase, negotiating what should be the network configuration in the end.
- In this process, the two main involved logics are the harmonization and translation, the former finds the set of intents compliant with all the defined requirements, and the latter takes care of the translation and enforcement.



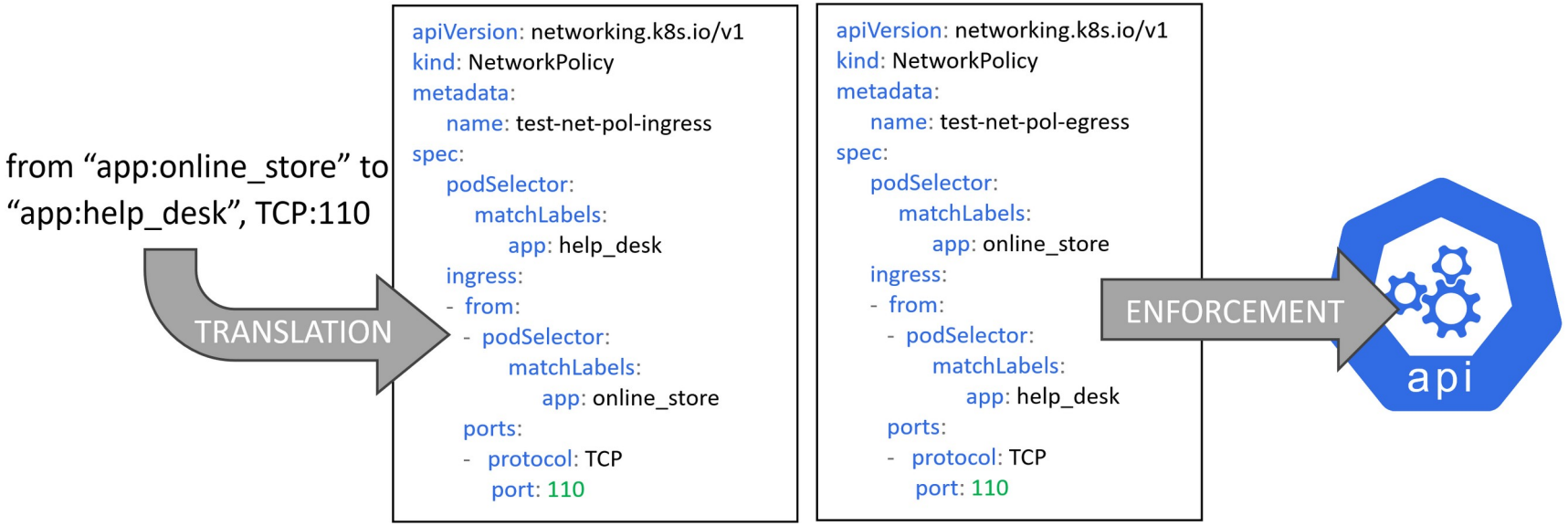
Harmonization

- An algorithm has been designed to perform the harmonization between different sets of intents established by users engaged in the resource acquisition process.
- The discordances that may arise can be classified in the following cases:
  - When a Request intent defined by the consumer is not (fully or partially) authorized by the Authorization intents of the provider.
  - When a MandatoryCommunication defined by the Provider is not satisfied by the intents of the Consumer.
  - When a Request intent defined by the consumer has been authorized but does not have a symmetrical Request intents in the provider.



Translation

- A module that translates the high-level intents, agnostic to the actual implementation, to the low-level configuration of Kubernetes Network Policies required to enforce the described network isolation.
- It is crucial to allow interoperability with multiple clusters that are possibly using multiple technologies (e.g., different CNIs and Network Policy formats)



Implementation

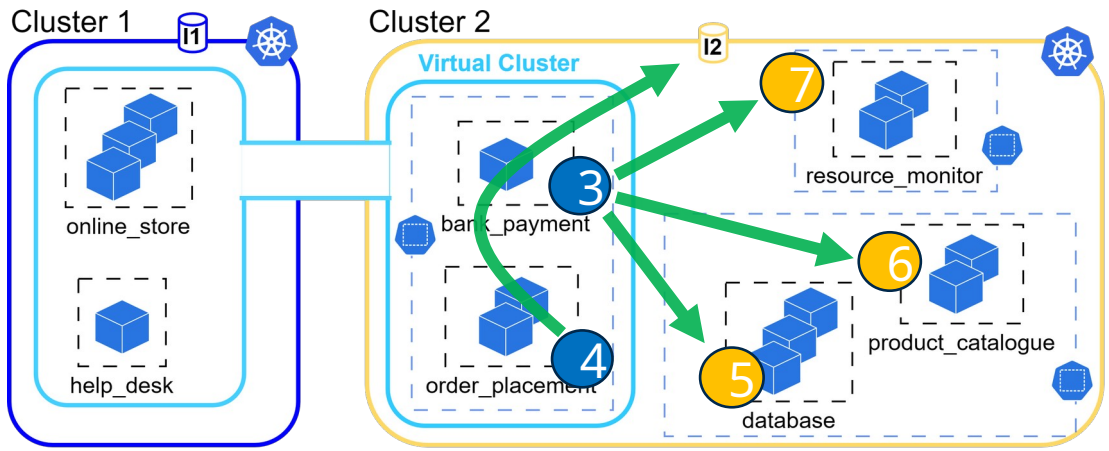
Implementation:

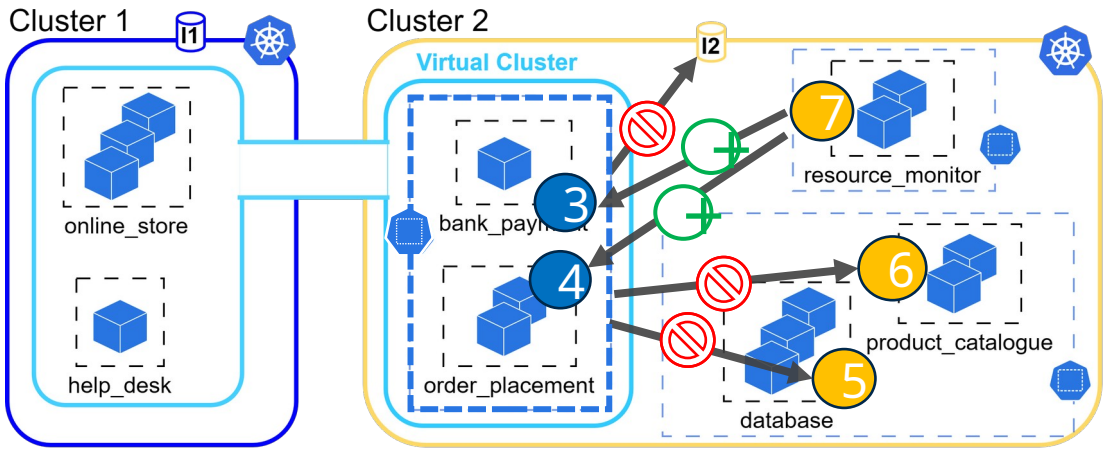
- Java-based framework, with REST APIs, that communicates and authenticates with the Kubernetes API server of different clusters.
- Translator module developed for the Calico CNI

Integration:

- European project FLUIDOS.

Validation use case






```
-----------------------------------------------------------------------------
[INFO]    List of harmonized Request intents (CONSUER) after type-1 resolution:
 (*) src:[ app:order_placement - name:fluidos ], sPort:[*],
      dst:[ app:product_catalogue - name:default ], dPort:[80], protocol:[TCP]
 (*) src:[ app:bank_payment - name:fluidos ], sPort:[*],
      dst:[128.0.0.0/1], dPort:[*], protocol:[ALL]
 (*) src:[ app:bank_payment - name:fluidos ], sPort:[*],
      dst:[64.0.0.0/2], dPort:[*], protocol:[ALL]
 (*) src:[ app:bank_payment - name:fluidos ], sPort:[*],
      dst:[32.0.0.0/3], dPort:[*], protocol:[ALL]
 (*) src:[ app:bank_payment - name:fluidos ], sPort:[*],
      dst:[16.0.0.0/4], dPort:[*], protocol:[ALL]
-----------------------------------------------------------------------------
```

Conclusions and Future Work

- A new approach has been proposed to automate the orchestration of **network isolation in a liquid computing environment**.
- By using user-defined intents the approach streamlines the configuration of network isolation primitives across multiple clusters, simplifying the implementation of common security patterns.
- Future work will cover the refinement of the security orchestrator capabilities, and its complete integration within the resource acquisition workflow currently under development within the FLUIDOS project.

Thanks for your attention!

Fulvio Valenza

<fulvio.valenza@polito.it>